ALL-IN-ONE-CONVERTER

PYTHON PROJECT

VITyarthi

PROJECT

NAME:DIVYA PAL

REG NO.:25BAI10788

1.Introduction

The ALL-IN-ONE-CONVERTER is a Python-based console application designed to provide users with a versatile tool for converting units across several domains, including length, mass, temperature, time, currency, and numerical bases. Its menu-driven approach offers a smooth and intuitive user experience for everyday and educational conversions.

2. Problem Statement

The abundance of single-purpose conversion tools leaves users switching between applications for different units. This project solves the problem by incorporating multiple converters into one easy-to-navigate interface, increasing convenience and efficiency.

3. Functional Requirements

- Convert length between meters and kilometers
- Convert temperature between Celsius and Fahrenheit
- Convert mass between kilograms and grams
- Convert duration between hours and minutes

- Convert currency between INR and USD (using fixed rates)
- Convert numbers between binary, octal, decimal, and hexadecimal
- Robust input validation and error handling

4. Non- functional Requirements

- Fast execution and immediate response
- Modular, maintainable code
- Clear prompts and console outputs
- Compatibility across platforms
- Minimal resource consumption

5. System Architecture

The program is designed as a linear, menu-driven command-line application. Individual conversion functions are called from a main loop that handles user navigation. No persistent storage is used.

6. Design Diagrams

- Use Case Diagram:

User interacts with the All-in-One-Converter to select conversion types and input data, receiving results.

- Workflow Diagram:

Start Show Main Menu Get User Choice Call Conversion Function Input Values Output Result Repeat/ Exit

- Sequence Diagram:

User Main Menu Converter Function Input Calculation Result Main Menu/Exit

- Class/Component Diagram:

Components: Main Menu, LengthConverter, TempConverter, MassConverter, DurationConverter, CurrencyConverter, BaseConverter

- ER Diagram:

Not applicable (no storage used)

7. Design Decisions & Rationale

- Seperation of logic into dedicated functions for readability and maintenance
- Linear menu design for ease of navigation

- Fixed currency rates to avoid network/API dependencies for demo
- Extensive input validation for reliability and user experience

8. Implementation Details

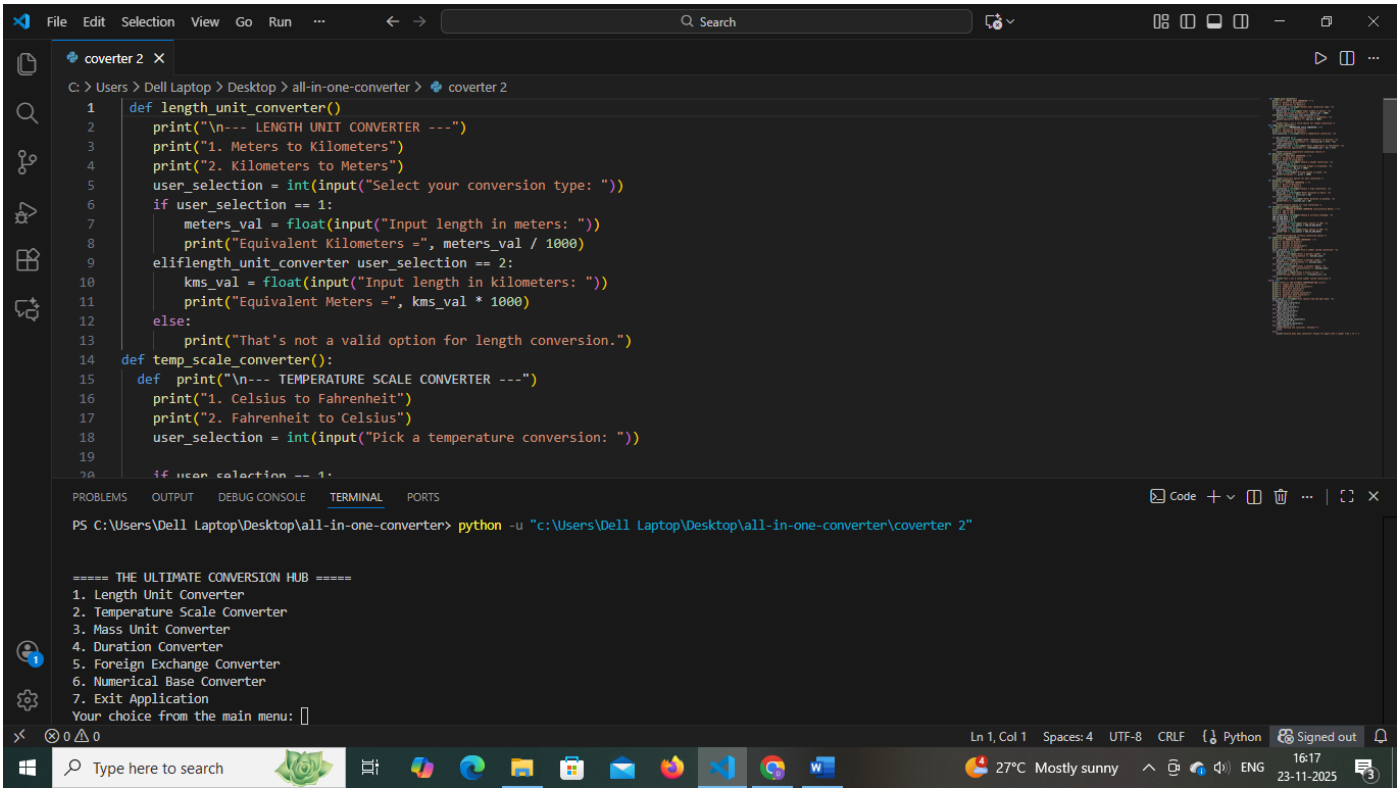
- Functions defined for each conversion type
- Main loop presents user menu and calls relevant function
- User input via console, conversion performed, results displayed
- Example:

```
def length_unit_converter():
    # ... function logic ...
```

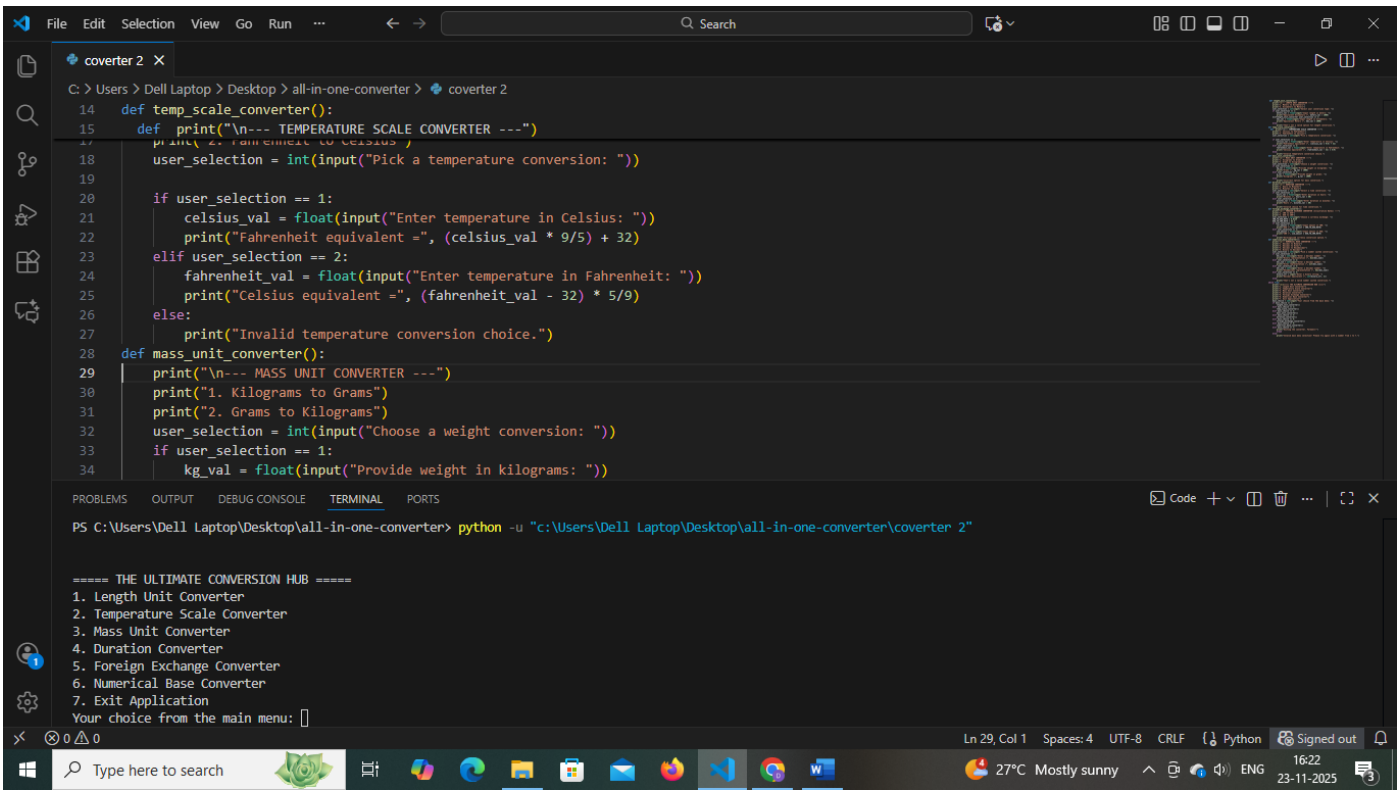
- Currency rates stored as constants

9. Screenshots / Results

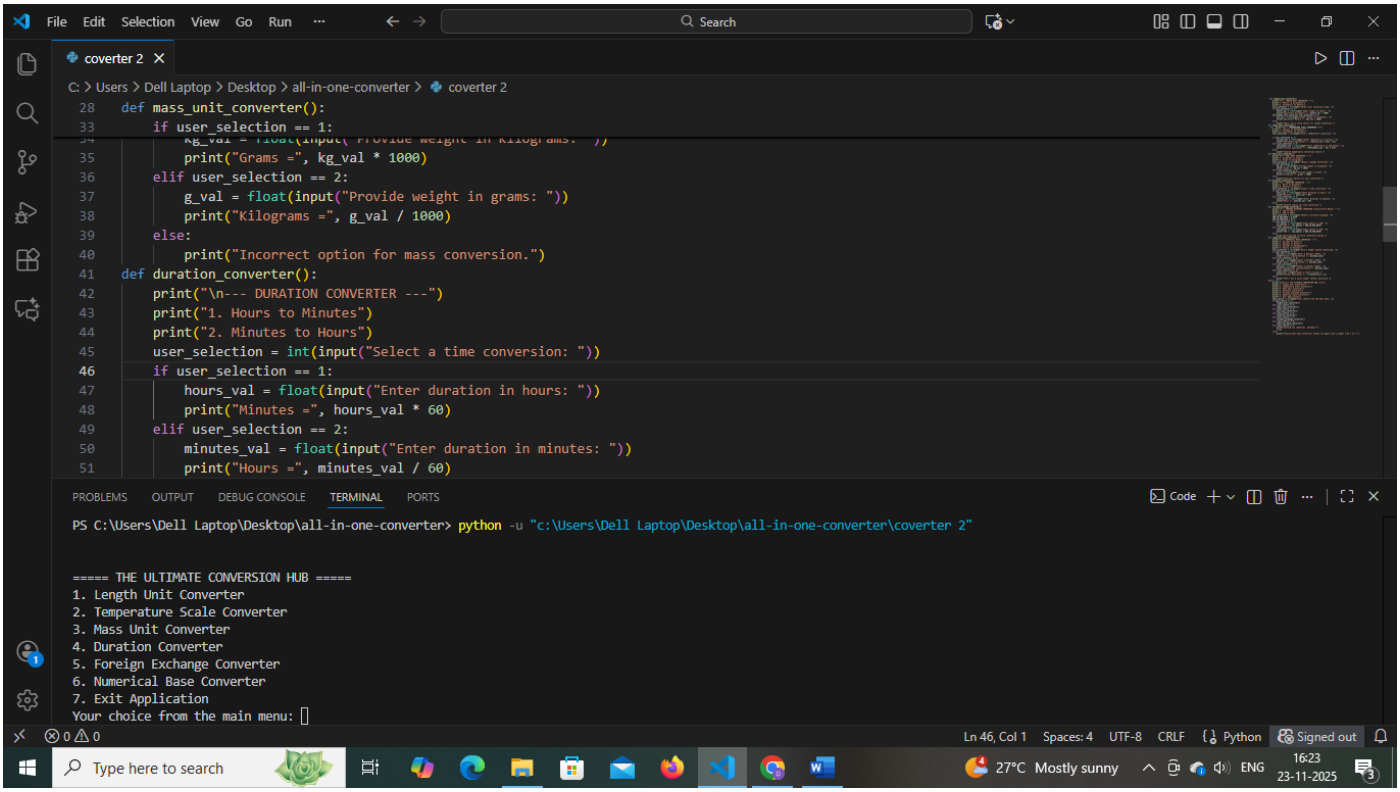
Screenshot 1:



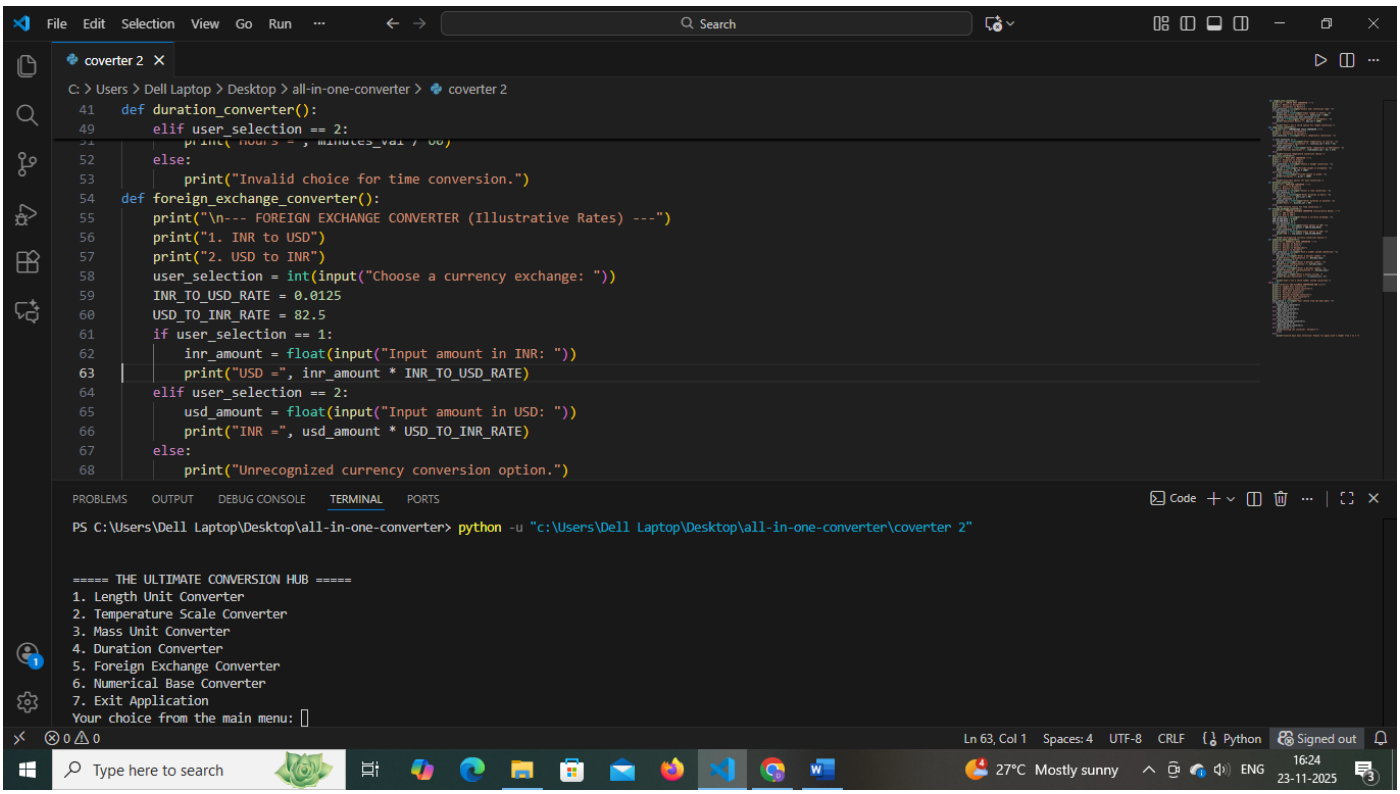
Screenshot 2:



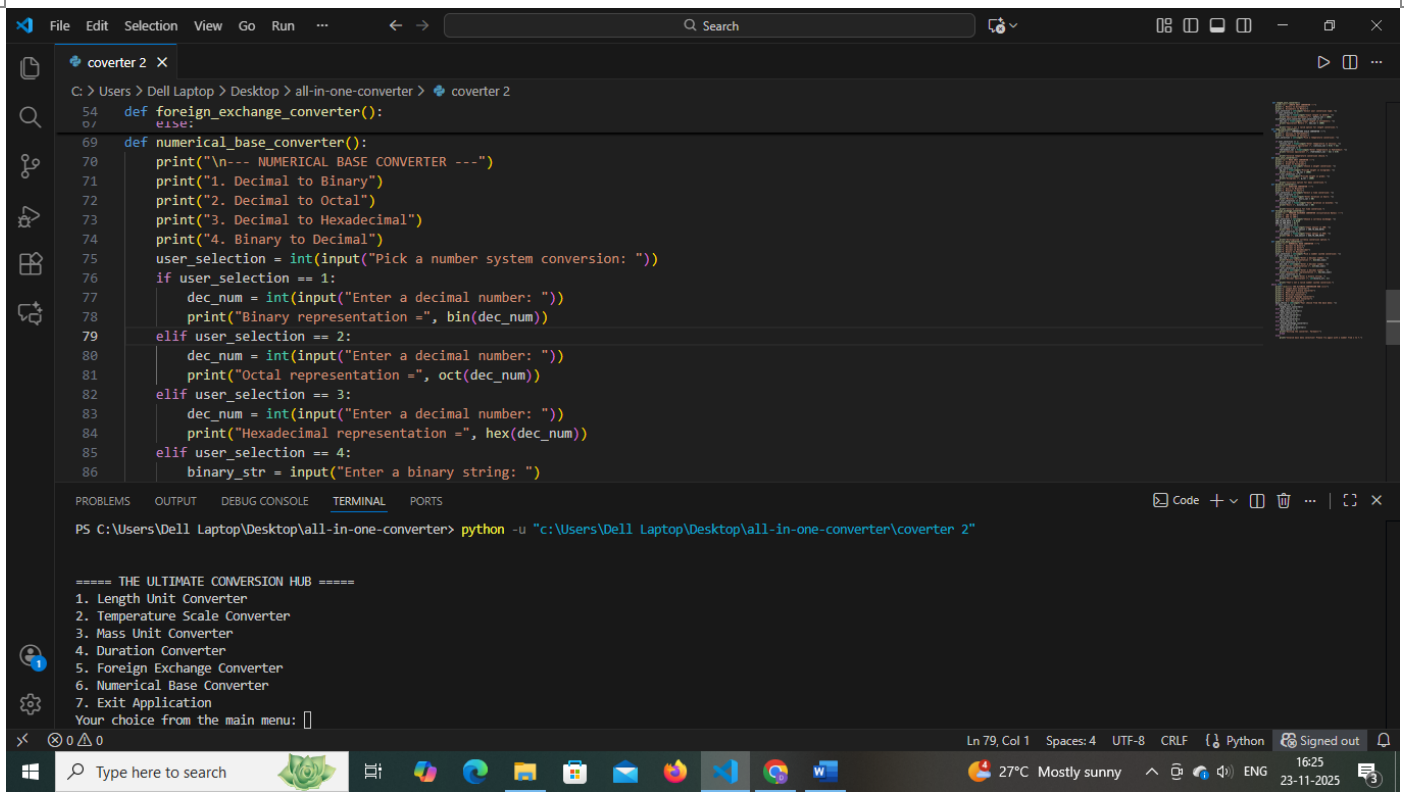
Screenshot 3:



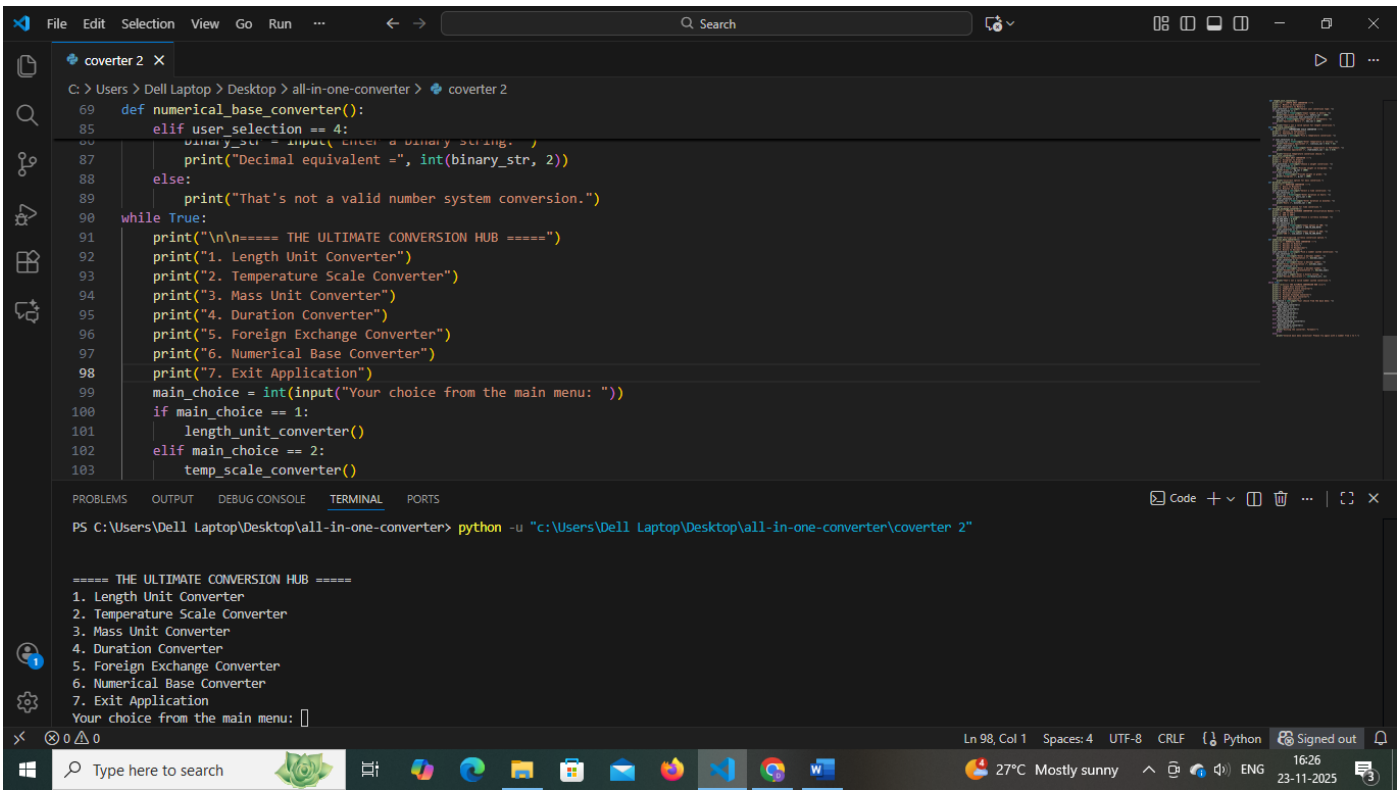
Screenshot 4:



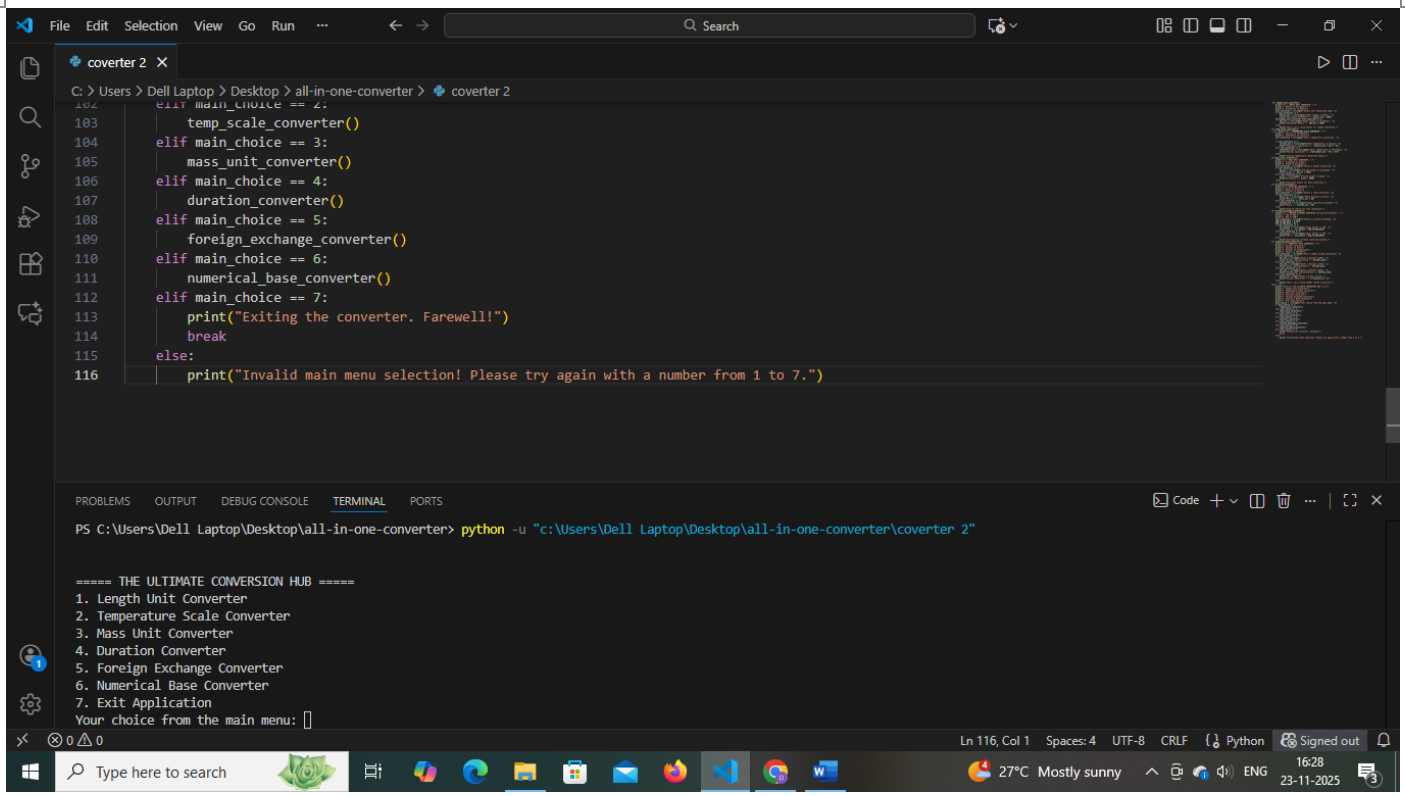
Screenshot 5:



Screenshot 6:



Screenshot 7:



OUTPUT

https://youtu.be/r8DxZiiPKDI?si=0-Go21zFjaUMSpal

10. Testing Approach

- Manual test for each function: valid/ invalid inputs
- Edge case testing for zero/negative values
- Output verification for each conversion type

11. Challenges Faced

- Comprehensive input validation

- Covering diverse conversion types in one application
- Streamlining user interface for clarity

12. Learnings & Key Takeaways

- Modular architecture aids scalability and maintenance
  - Clear documentation increases usability
  - Input checking is essential for preventing errors and ensuring user trust

13. Future Enhancements

- Add persistent storage for history
- Integrate real-time exchange rates via API
- Add volume, speed, and additional conversions
- Develop a graphical user interface

- Implement automated testing framework

14. References

- Swimm Documentation: Python Best Practices
- Docuwriter Python Documentation Guide
- Dagster Blog: Structuring Python Projects
- PEP 8 – Style Guide for Python Code
- Real Python: Documenting Python Code